

AEGISPAY — SELL

The AI buyer checkout (simplified)

How a customer safely buys through an AI — in one minute.

The one-minute flow

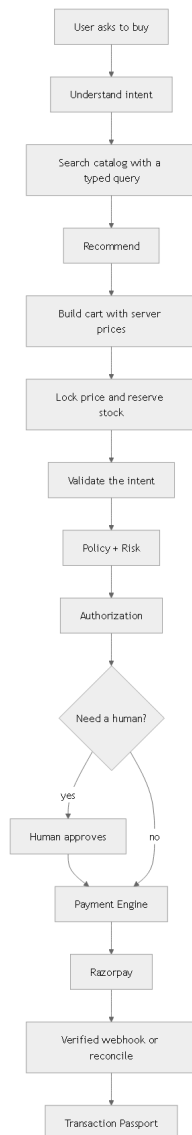


Figure 1 — From 'I want running shoes' to a verified purchase

- 1 **User asks.** “Find running shoes under ■4,000.”
- 2 **Understand intent.** The AI turns it into a typed, structured intent (product, budget, limits).
- 3 **Search catalog.** A real query against the merchant's products.
- 4 **Recommend.** The best few, priced by the store.
- 5 **Build cart.** Prices are server-owned; the AI can't change them.
- 6 **Lock price + reserve stock.** A price version is captured and inventory is held.

- 7 **Validate intent.** Strictly checked — nothing malformed gets through.
- 8 **Policy + Risk.** Limits and an explainable risk score.
- 9 **Authorization.** Scoped, expiring, bound to this one purchase.
- 10 **Human if needed.** High value → a person approves first.
- 11 **Payment Engine** → **Razorpay.** Only approved money moves.
- 12 **Verified webhook / reconcile.** Provider truth resolves the outcome.
- 13 **Transaction Passport.** Full, verifiable proof of the purchase.

The core rule: **the AI reasons and proposes; AegisPay decides; Razorpay executes only approved actions.**

Key safety details

What	What it means
Typed intent	The AI's output is a strict, validated schema — not free text.
Server-owned prices	Price comes from the database, never from the AI or the browser.
Cart hash	A snapshot of the cart; any change invalidates the authorization.
Price version + inventory reserve	A price change is caught; stock is held so it can't oversell.
Cart expiry	A stale cart expires — you can't pay an old quote.
Idempotency	Same request twice = one result. Designed to prevent duplicate charges.
Scoped/expiring authorization	Bound to one purchase, expires, single-use.
Agent identity	The agent is authenticated and scoped; it can't impersonate or elevate.
Prompt injection defense	Catalog is data, not instructions; the intent is validated.
Failure recovery	Timed-out payments are reconciled, never blindly retried.
Refund/cancellation	Safe, idempotent, capped to what was captured.
Audit trail	Every step is recorded tamper-evidently.

Payment safety (the important part)

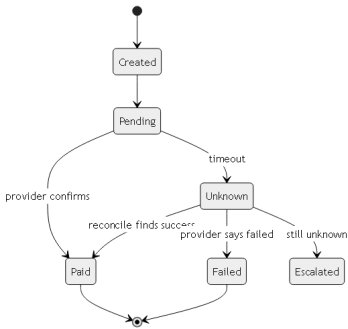


Figure 2 — The payment lifecycle, including the UNKNOWN recovery path

if Razorpay is slow, the payment becomes UNKNOWN. We do not retry — that could charge twice. We check with the provider, then finish or fail safely. This is **designed to prevent duplicate charges**.

Refund & cancellation

Refunds are idempotent and capped to the captured amount.

Cancellation only from an allowed state, and it is audited.

A person can always review and reverse an AI-driven action.

Why AegisPay is difficult to reject

The AI can reason and recommend, but only AegisPay's deterministic control plane can authorize and execute financial actions. That is the whole product.

It is safe by design, not by promise. The AI has no money keys and no payment endpoint — only a request that is always checked.

It handles failure honestly. Unknown payments are reconciled rather than blindly retried; duplicates are designed out, not hoped away.

You can prove it. Every decision is written to a tamper-evident audit ledger and a Transaction Passport.

• **It is simple to run.** One FastAPI service, PostgreSQL + Redis + one queue on AWS ECS — no unnecessary complexity.

• **It grows and sells.** GROW lifts revenue with bounded risk; SELL lets AI buyers transact end-to-end. PROTECT keeps both safe.